Artificial Intelligence

AI-2002

Lab 11

Unsupervised Learning

National University of Computer & Emerging Sciences – NUCES – Karachi

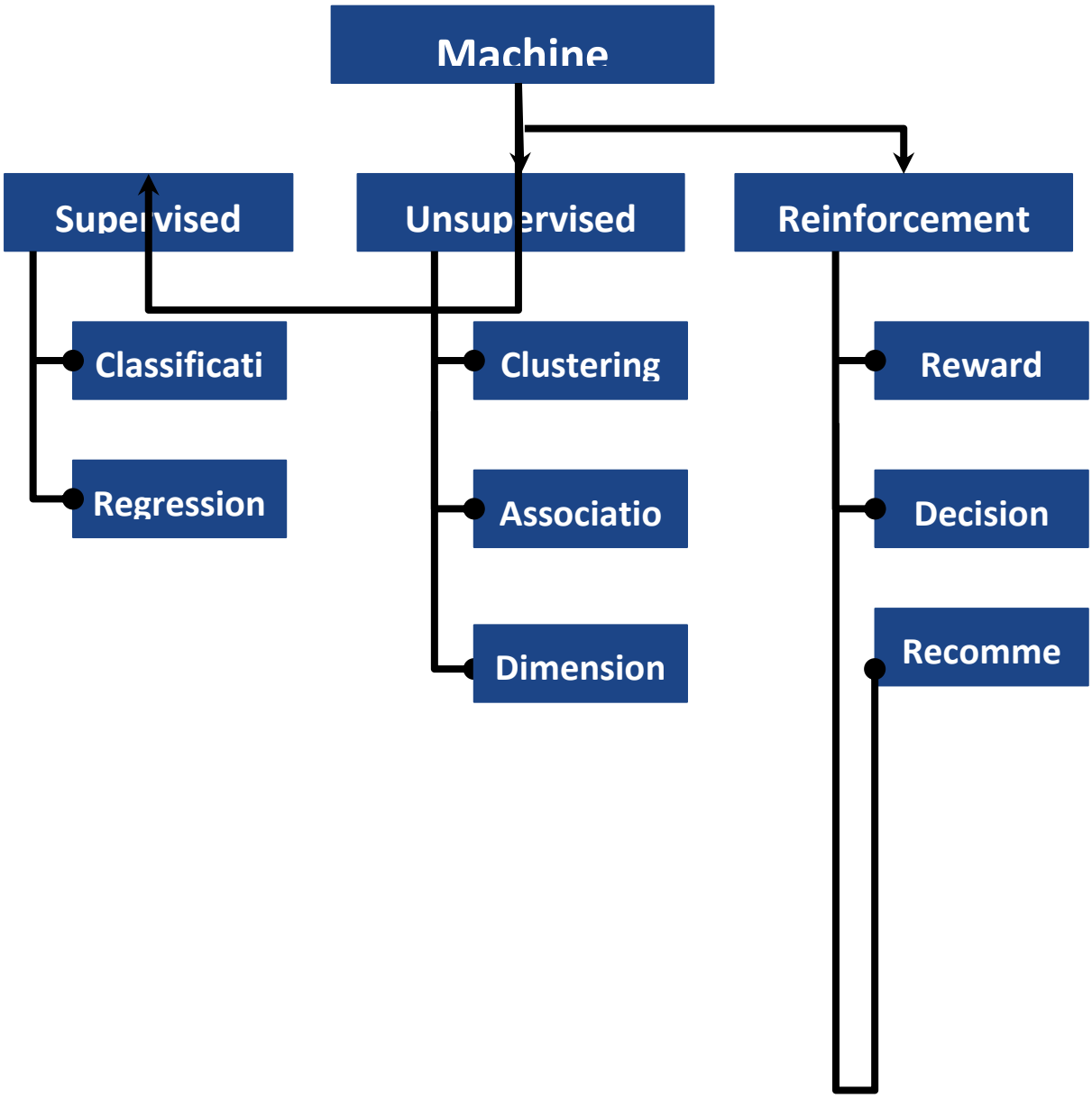
Course Code: AI-2002	Artificial Intelligence Lab

1. Objective

1. The fundamentals of machine learning and its various types.
2. How supervised learning differs from other types of machine learning.
3. The usage of Python libraries like NumPy, Pandas, Matplotlib, and Scikit-learn for implementing machine learning algorithms.
4. Practical implementation of supervised learning classification algorithms such as Linear Regression, Logistic Regression and SVM.
5. Some examples of regression algorithms like Decision Trees.

2. Machine Learning and Its Types

Machine learning is a subset of **artificial intelligence** that enables systems to learn from data and improve their performance without explicit programming. It is broadly categorized into three types:



2.1 Libraries Used in Machine Learning

Below mentioned Python libraries are commonly used to implement machine learning algorithms:

1. **NumPy**: Provides support for numerical computations and working with arrays.
2. **Pandas**: Facilitates data manipulation and analysis.
3. **Matplotlib**: Used for data visualization.

4. **Scikit-learn**: A comprehensive library for implementing both supervised and unsupervised learning algorithms. It also provides tools for data preprocessing, model selection, and evaluation metrics.
5. **Seaborn**: Enhances data visualization by providing advanced plotting functions.

3. Introduction to Unsupervised Learning

Unsupervised Learning is a type of machine learning where the model is not given any labeled data. Instead of being told what the correct output is, the model tries to find patterns or structure in the data on its own. The algorithm learns from the data itself — without guidance or answers.

Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format.

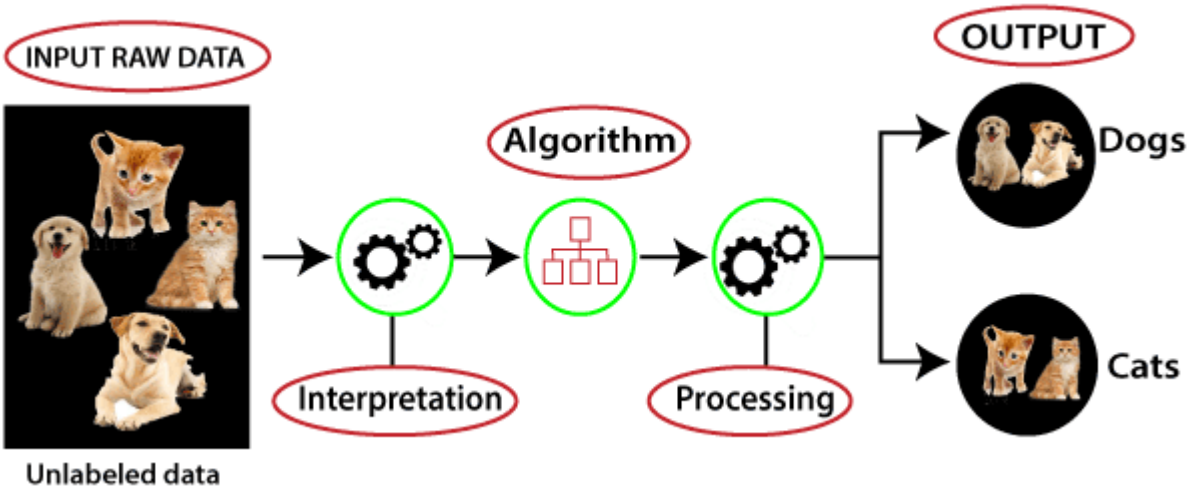
Example: Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.

Why use Unsupervised Learning?

Below are some main reasons which describe the importance of Unsupervised Learning:

- Unsupervised learning is helpful for finding useful insights from the data.
- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- Unsupervised learning works on unlabeled and uncategorized data which make unsupervised learning more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.

Working of Unsupervised Learning



3.1 Applications of Unsupervised Learning

- Customer Segmentation
- Document or News Clustering
- Music or Movie Recommendation

3.2 Types of Unsupervised Learning

There are three types of Unsupervised learning algorithms defined as follows:

1. **Clustering:** Clustering is a way of grouping similar things together. In machine learning and data science, clustering is used to group similar data points into clusters, so that items in the same cluster are more alike than those in other clusters. It is an unsupervised learning technique, which means it works without labeled data.

2. **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occur together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.
3. **Dimensionality Reduction:** Dimensionality Reduction is the process of reducing the number of features (columns) in your dataset while keeping the important information. It helps simplify complex data, makes it easier to visualize, and often speeds up machine learning algorithms.

4. Clustering

4.1 K-Means clustering

K-Means Clustering is an Unsupervised Learning algorithm, which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if K=2, there will be two clusters, and for K=3, there will be three clusters, and so on.

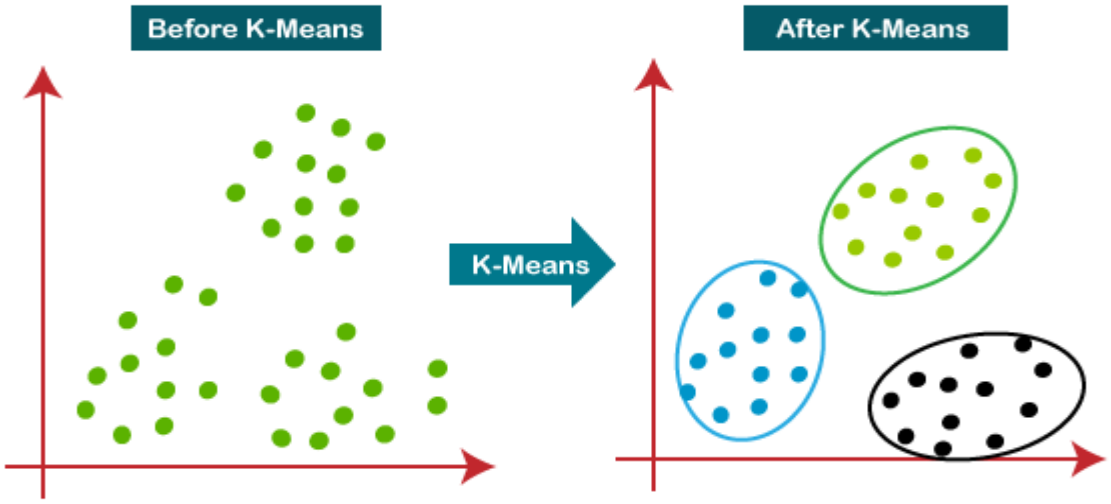
It is an iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs to only one group that has similar properties.

The k-means clustering algorithm mainly performs two tasks:

- Determines the best value for K center points or centroids by an iterative process.
- Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.

Hence each cluster has datapoints with some commonalities, and it is away from other clusters.

The below diagram explains the working of the K-means Clustering Algorithm:



Elbow Method: The Elbow method is one of the most popular ways to find the optimal number of clusters. This method uses the concept of WCSS value. WCSS stands for Within Cluster Sum of Squares, which defines the total variations within a cluster. The formula to calculate the value of WCSS (for 3 clusters) is given below:

WCSS= ∑Pi in Cluster1 distance(Pi C1)2 +∑Pi in Cluster2distance(Pi C2)2+∑Pi in CLuster3 distance(Pi C3)2

In the above formula of WCSS,

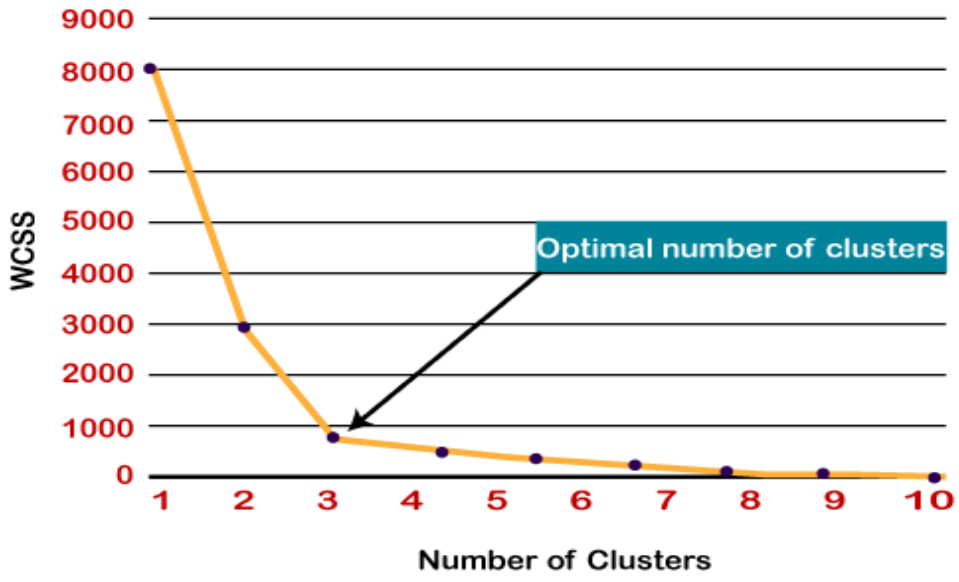
∑Pi in Cluster1 distance(Pi C1)2: It is the sum of the square of the distances between each data point and its centroid within a cluster1 and the same for the other two terms.

To measure the distance between data points and centroid, we can use any method such as Euclidean distance or Manhattan distance.

To find the optimal value of clusters, the elbow method follows the below steps:

- It executes the K-means clustering on a given dataset for different K values (ranges from 1-10).
- For each value of K, calculates the WCSS value.
- Plots a curve between calculated WCSS values and the number of clusters K.
- The sharp point of bend or a point of the plot looks like an arm, then that point is considered as the best value of K.

Since the graph shows the sharp bend, which looks like an elbow, hence it is known as the elbow method. The graph for the elbow method looks like the below image:



4.1.1 How does the K-means algorithm works

Step-1: Select the number K to decide the number of clusters.
Step-2: Select random K points or centroids. (It can be different from the input dataset).
Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.
Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.
Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.
Step-7: The model is ready.

4.1.2 HOW TO IMPLEMENT K-means clustering?

```
#importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler



# Importing the dataset
df = pd.read_csv('Mall_Customers.csv')
df.head()

# Extracting Variables
x = df.iloc[:, [3, 4]].values

#finding optimal number of clusters using the elbow method
from sklearn.cluster import KMeans
wcss_list= []  #Initializing the list for the values of WCSS

#Using for loop for iterations from 1 to 10.
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state= 42)
    kmeans.fit(x)
    wcss_list.append(kmeans.inertia_)
mtp.plot(range(1, 11), wcss_list)
mtp.title('The Elobw Method Graph')
mtp.xlabel('Number of clusters(k)')
mtp.ylabel('wcss_list')
mtp.show()
```

```
# Normalize features for better clustering
scaler = StandardScaler()
X_scaled = scaler.fit_transform(x)

#training the K-means model on a dataset
kmeans = KMeans(n_clusters=5, init='k-means++', random_state= 42)
y_predict= kmeans.fit_predict(X_scaled)


#visulaizing the clusters
mtp.scatter(x[y_predict == 0, 0], x[y_predict == 0, 1], s = 100, c =
'blue', label = 'Cluster 1') #for first cluster
mtp.scatter(x[y_predict == 1, 0], x[y_predict == 1, 1], s = 100, c =
'green', label = 'Cluster 2') #for second cluster
mtp.scatter(x[y_predict== 2, 0], x[y_predict == 2, 1], s = 100, c =
'red', label = 'Cluster 3') #for third cluster
mtp.scatter(x[y_predict == 3, 0], x[y_predict == 3, 1], s = 100, c =
'black', label = 'Cluster 4') #for fourth cluster
mtp.scatter(x[y_predict == 4, 0], x[y_predict == 4, 1], s = 100, c =
'purple', label = 'Cluster 5') #for fifth cluster
mtp.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:,
1], s = 300, c = 'yellow', label = 'Centroid')
mtp.title('Clusters of customers')
mtp.xlabel('Annual Income (k$)')
mtp.ylabel('Spending Score (1-100)')
mtp.legend()
mtp.show()
```
